

ASSIGNMENT 15.3

K.Hithesh

2303A51291

Prompt:

Prompt:

Act as a senior backend developer and generate a fully working REST API for a **Real-Time Online Food Ordering System**.

Requirements:

Build the API using **Python (Flask or FastAPI preferred)** with clean, modular, and well-documented code.

Core Features:

Implement the following endpoints:

1. **GET /menu**
 - Return a list of available food items
 - Each item should include: id, name, price, availability
2. **POST /order**
 - Place a new order
 - Accept JSON input with items and quantities
 - Generate a unique order ID
 - Store order with status: "Placed"
3. **GET /order/{id}**
 - Fetch details of a specific order
 - Include items, total price, and current status
4. **PUT /order/{id}**
 - Update an existing order (modify items or quantities)
 - Recalculate total price
5. **DELETE /order/{id}**
 - Cancel an order
 - Update status to "Cancelled"

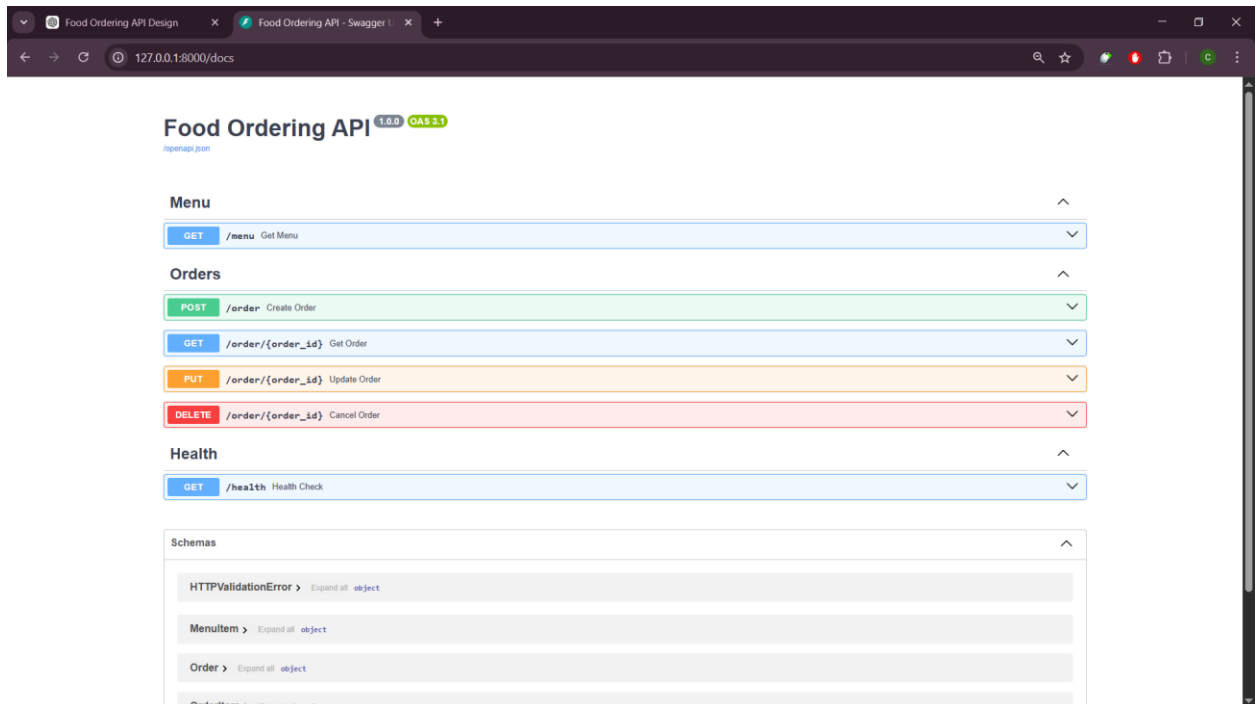
Code:

```
1 from http.server import BaseHTTPRequestHandler, HTTPServer
2 import json
3 from urllib.parse import urlparse
4
5 # Sample menu
6 menu = [
7     {"id": 1, "name": "Pizza", "price": 250, "available": True},
8     {"id": 2, "name": "Burger", "price": 120, "available": True},
9     {"id": 3, "name": "Pasta", "price": 180, "available": True}
10 ]
11
12 orders = {}
13 order_id_counter = 1
14
15
16 class FoodAPI(BaseHTTPRequestHandler):
17
18     def _set_headers(self, code=200):
19         self.send_response(code)
20         self.send_header('content-type', 'application/json')
21         self.end_headers()
22
23     def do_GET(self):
24         parsed = urlparse(self.path)
25         path = parsed.path
26
27         # GET /menu
28         if path == "/menu":
29             self._set_headers()
30             self.wfile.write(json.dumps(menu).encode())
31
32         # GET /order/{id}
33         elif path.startswith("/order/"):
34             order_id = path.split("/")[-1]
35             if order_id in orders:
36                 self._set_headers()
37                 self.wfile.write(json.dumps(orders[order_id]).encode())
38             else:
39                 self._set_headers(404)
40                 self.wfile.write(json.dumps({"error": "Order not found"}).encode())
```

```
16 class FoodAPI(BaseHTTPRequestHandler):
23     def do_GET(self):
24         self._set_headers(404)
25         self.wfile.write(json.dumps({"error": "Order not found"}).encode())
26
27     else:
28         self._set_headers(404)
29         self.wfile.write(json.dumps({"error": "Invalid endpoint"}).encode())
30
31     def do_POST(self):
32         global order_id_counter
33
34         if self.path == "/order":
35             content_length = int(self.headers['Content-Length'])
36             body = self.rfile.read(content_length)
37             data = json.loads(body)
38
39             items = data.get("items", [])
40             if not items:
41                 self._set_headers(400)
42                 self.wfile.write(json.dumps({"error": "No items provided"}).encode())
43                 return
44
45             total = 0
46             for item in items:
47                 for m in menu:
48                     if m["id"] == item["id"]:
49                         total += m["price"] * item["quantity"]
50
51             order_id = str(order_id_counter)
52             order_id_counter += 1
53
54             orders[order_id] = {
55                 "id": order_id,
56                 "items": items,
57                 "total": total,
58                 "status": "Placed"
59             }
```

Output:

```
ready = selector.select(poll_interval)
File "C:\Users\Administrator\AppData\Local\Python\pythoncore-3.14-64\lib\selectors.py", line 314, in select
    r, w, _ = self._select(self._readers, self._writers, [], timeout)
File "C:\Users\Administrator\AppData\Local\Python\pythoncore-3.14-64\lib\selectors.py", line 305, in _select
    r, w, x = select.select(r, w, w, timeout)
KeyboardInterrupt
PS C:\Users\Administrator\Desktop\aiasscode> python -m uvicorn app:app --reload
INFO: Will watch for changes in these directories: ['C:\Users\Administrator\Desktop\aiasscode']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reload process [3672] using StatReload
INFO: Started server process [11788]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:61263 - "GET / HTTP/1.1" 404 Not Found
```



Explanation:

an Online Food Ordering API is created where a menu of food items is maintained, and users can place orders, view order details, update orders, and cancel them. The system calculates the total price based on selected items and maintains an order status, showing how CRUD operations are applied in an e-commerce-like environment

TASK2:

PROMPT:

Prompt:

Act as a backend developer and create a REST API for an **Employee Payroll System** using **Python (FastAPI preferred)** in one file.

Endpoints:

- GET /employees → list all employees
- POST /employees → add employee (name, role, salary)
- PUT /employees/{id}/salary → update employee salary
- DELETE /employees/{id} → delete employee

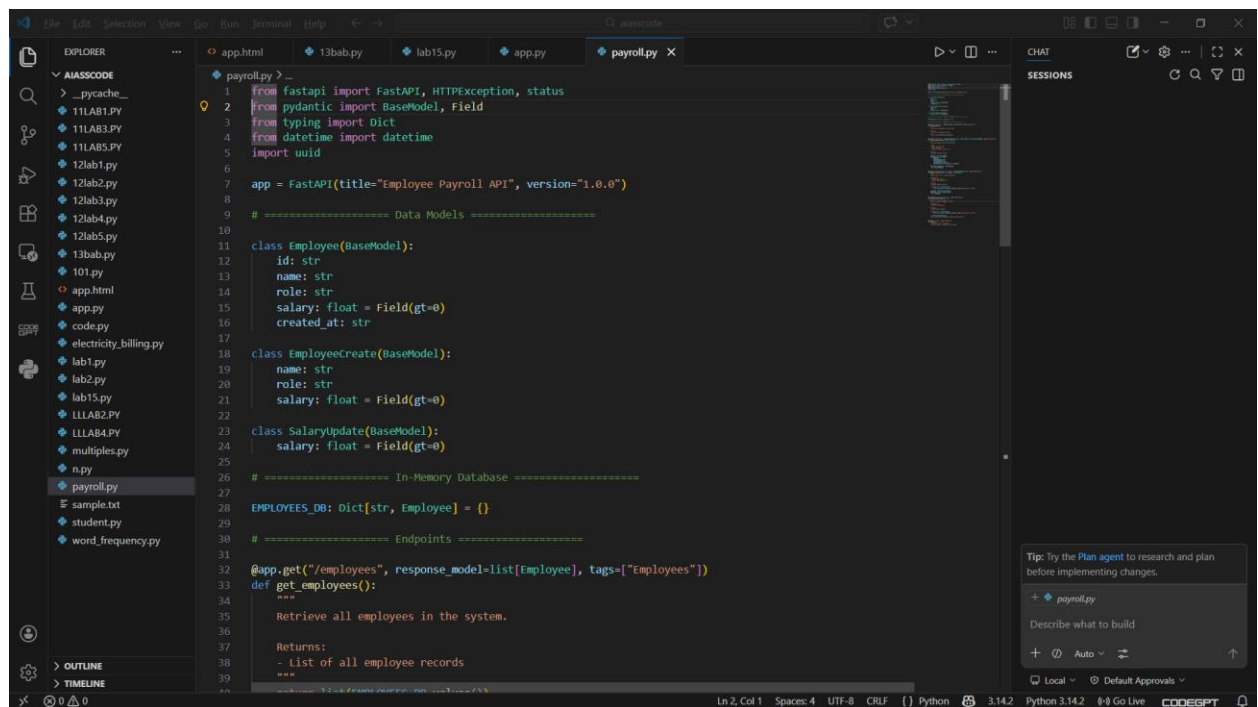
Requirements:

- Use in-memory storage (dict/list)
- Create proper data models (Employee, SalaryUpdate)
- Validate salary (> 0)
- Use correct HTTP status codes
- Add comments/docstrings for each endpoint

Output:

- Full working code
- Run instructions (uvicorn)
- Sample requests
- Suggested improvements (auth, DB, logging, pagination)

CODE:



```
1 from fastapi import FastAPI, HTTPException, status
2 from pydantic import BaseModel, Field
3 from typing import Dict
4 from datetime import datetime
5 import uuid
6
7 app = FastAPI(title="Employee Payroll API", version="1.0.0")
8
9 # ===== Data Models =====
10
11 class Employee(BaseModel):
12     id: str
13     name: str
14     role: str
15     salary: float = Field(gt=0)
16     created_at: str
17
18 class EmployeeCreate(BaseModel):
19     name: str
20     role: str
21     salary: float = Field(gt=0)
22
23 class SalaryUpdate(BaseModel):
24     salary: float = Field(gt=0)
25
26 # ===== In-Memory Database =====
27 EMPLOYEES_DB: Dict[str, Employee] = {}
28
29 # ===== Endpoints =====
30
31 @app.get("/employees", response_model=list[Employee], tags=["Employees"])
32 def get_employees():
33     """
34     Retrieve all employees in the system.
35
36     Returns:
37     - list of all employee records
38     """
```

The screenshot shows a VS Code editor window with the file explorer on the left displaying a project named 'AIASSCODE'. The 'payroll.py' file is open in the editor. The code defines two functions: `get_employees()` and `add_employee()`. `get_employees()` returns a list of employee records from `EMPLOYEES_DB.values()`. `add_employee()` is a Flask endpoint that takes an employee object and adds it to the database. It includes input parameters like name, role, and salary, and returns the created employee object. The status code is `HTTP_201_CREATED` and the tag is 'Employees'. The database is `EMPLOYEES_DB`. The status bar at the bottom indicates 'Ln 47, Col 5', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', '3.14.2', 'Python 3.14.2', 'Go Live', and 'CODESPOT'.

```
def get_employees():
    """
    Returns:
    - List of all employee records
    """
    return list(EMPLOYEES_DB.values())

@app.post("/employees", response_model=Employee, status_code=status.HTTP_201_CREATED, tags=["Employees"])
def add_employee(employee: EmployeeCreate):
    """
    Add a new employee with salary details.

    Input:
    - name: Employee name
    - role: Job role
    - salary: Initial salary (> 0)

    Returns:
    - Created employee object
    """
    emp_id = str(uuid.uuid4())
    new_employee = Employee(
        id=emp_id,
        name=employee.name,
        role=employee.role,
        salary=employee.salary,
        created_at=datetime.utcnow().isoformat()
    )
    EMPLOYEES_DB[emp_id] = new_employee
    return new_employee

@app.put("/employees/{emp_id}/salary", response_model=Employee, tags=["Employees"])
def update_salary(emp_id: str, update: SalaryUpdate):
    """
    Update salary of an existing employee.

    Parameters:
    - emp_id: Employee ID
    - update: SalaryUpdate object
    """
```

The screenshot shows the same VS Code editor window with 'payroll.py' open. The code continues with a `delete_employee()` function and a `health()` endpoint. `delete_employee()` is a Flask endpoint that takes an employee ID and removes the employee from the database. It includes a parameter `emp_id` and returns a confirmation message. The status code is `HTTP_200_OK` and the tag is 'Employees'. The database is `EMPLOYEES_DB`. The `health()` endpoint is a GET request that returns the status 'Payroll API running'. The status bar at the bottom indicates 'Ln 102, Col 29', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', '3.14.2', 'Python 3.14.2', 'Go Live', and 'CODESPOT'.

```
def delete_employee(emp_id: str):
    """
    Remove an employee from the system.

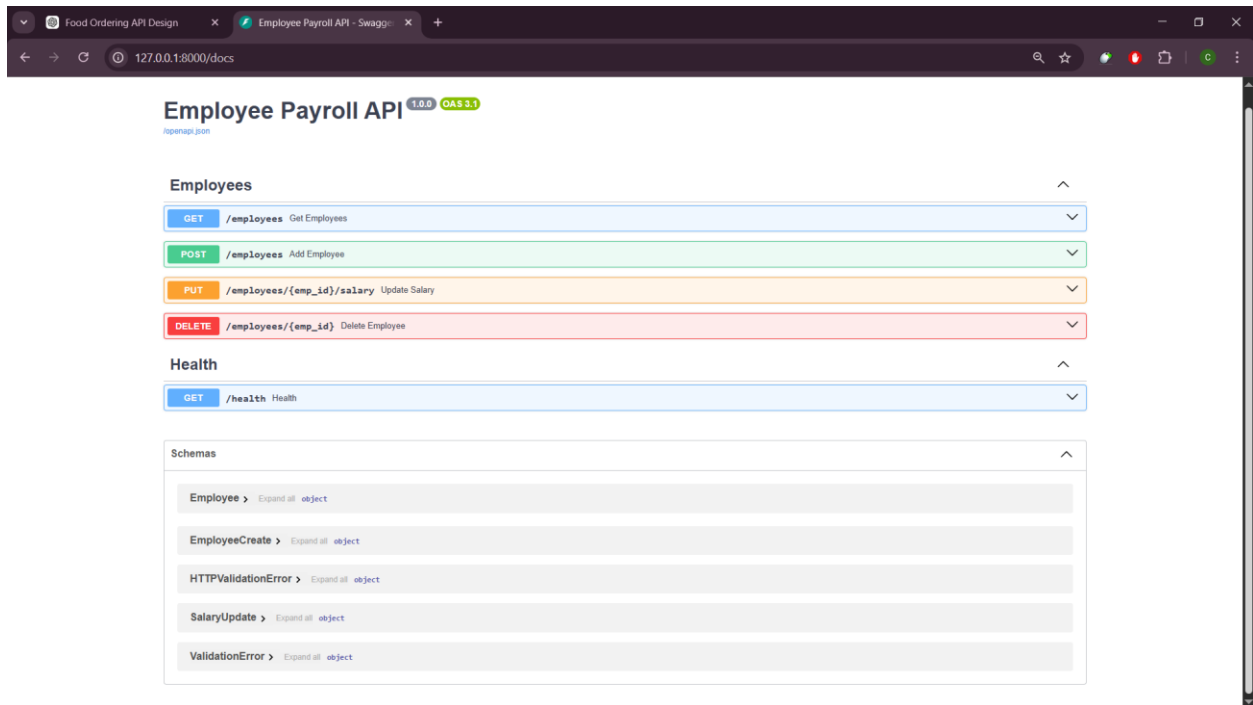
    Parameters:
    - emp_id: Employee ID

    Returns:
    - Confirmation message
    """
    if emp_id not in EMPLOYEES_DB:
        raise HTTPException(status_code=404, detail="Employee not found")

    del EMPLOYEES_DB[emp_id]
    return {"message": "Employee deleted successfully"}

@app.get("/health", tags=["Health"])
def health():
    """Check if API is running"""
    return {"status": "Payroll API running"}
```

OUTPUT:



Explanation:

an Online Food Ordering API is created where a menu of food items is maintained, and users can place orders, view order details, update orders, and cancel them. The system calculates the total price based on selected items and maintains an order status, showing how CRUD operations are applied in an e-commerce-like environment

Task3:

Prompt:

Prompt:

Act as a backend developer and create a REST API for a **Student Information Management System** using **Python (FastAPI preferred)** in one file.

Features:

- Add a new student (name, roll number, course, year)
- Get all students
- Update student details
- Delete a student

Endpoints:

- GET /students → list all students
- POST /students → add new student

- PUT /students/{id} → update student info
- DELETE /students/{id} → delete student

Requirements:

- Use in-memory storage (dictionary/list)
- Design proper data models (Student, StudentCreate)
- Validate inputs (year > 0, roll number unique)
- Use proper HTTP status codes
- Add comments/docstrings for each endpoint

Output:

- Full working code
- Run instructions (uvicorn)
- Sample API requests
- Suggested improvements (authentication, database, pagination, search)

Code:

```

1 from fastapi import FastAPI, HTTPException, status
2 from pydantic import BaseModel, Field
3 from typing import Dict
4 import uuid
5
6 app = FastAPI(title="Student Management API", version="1.0.0")
7
8 # ===== Data Models =====
9
10 class Student(BaseModel):
11     id: str
12     name: str
13     roll_number: str
14     course: str
15     year: int = Field(gt=0, le=5)
16
17 class StudentCreate(BaseModel):
18     name: str
19     roll_number: str
20     course: str
21     year: int = Field(gt=0, le=5)
22
23 # ===== In-Memory Database =====
24
25 STUDENTS_DB: Dict[str, Student] = {}
26
27 # ===== Endpoints =====
28
29 @app.get("/students", response_model=list[Student], tags=["Students"])
30 def get_students():
31     """
32     Get all students
33     """
34     return list(STUDENTS_DB.values())
35
36
37 @app.post("/students", response_model=Student, status_code=status.HTTP_201_CREATED, tags=["Students"])
38 def add_student(student: StudentCreate):
39     """
40     Add a new student
41     """

```

```
def add_student(student: StudentCreate):
    """
    Add a new student
    """
    STUDENTS_DB[student_id] = new_student
    return new_student

@app.put("/students/{student_id}", response_model=Student, tags=["Students"])
def update_student(student_id: str, student: StudentCreate):
    """
    Update student details
    """
    if student_id not in STUDENTS_DB:
        raise HTTPException(status_code=404, detail="Student not found")

    updated_student = Student(
        id=student_id,
        name=student.name,
        roll_number=student.roll_number,
        course=student.course,
        year=student.year
    )

    STUDENTS_DB[student_id] = updated_student
    return updated_student

@app.delete("/students/{student_id}", tags=["Students"])
def delete_student(student_id: str):
    """
    Delete a student record
    """
    if student_id not in STUDENTS_DB:
        raise HTTPException(status_code=404, detail="Student not found")

    del STUDENTS_DB[student_id]
    return {"message": "Student deleted successfully"}
```

Output:

```
PS C:\Users\Administrator\Desktop\aiasscode> & C:/Users/Administrator/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/Users/Administrator/Desktop/aiasscode/rest.py
PS C:\Users\Administrator\Desktop\aiasscode> python -m uvicorn app:app --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\Administrator\\Desktop\\aiasscode']
INFO: Uvicorn running on http://127.0.0.1:8000 (press CTRL+C to quit)
INFO: Started reloader process [3392] using StatReload
INFO: Started server process [416]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Student Management API 1.0.0 OpenAPI 3.1

default

- GET / Home

Students

- GET /students Get Students
- POST /students Add Student
- PUT /students/{student_id} Update Student
- DELETE /students/{student_id} Delete Student

Health

- GET /health Health

Schemas

- HTTPValidationError > Expand all object
- Student > Expand all object
- StudentCreate > Expand all object
- ValidationError > Expand all object

Explanation:

a Student Information API is developed to manage student records such as name, roll number, course, and year. It ensures unique roll numbers and provides endpoints to add, view, update, and delete student details, representing a university database system.

Task4:

Prompt:

Prompt:

Act as a backend developer and build a REST API for a **Weather Information Service** using **Python (FastAPI preferred)**.

Endpoints:

- GET /weather/current/{city} → fetch current weather
- GET /weather/forecast/{city} → fetch 5-day forecast

Requirements:

- Use a public API (OpenWeatherMap)
- Fetch real-time data using HTTP requests
- Return structured JSON with:
 - temperature
 - humidity
 - weather condition
- Handle errors (invalid city, API failure)
- Use environment variable for API key

Output:

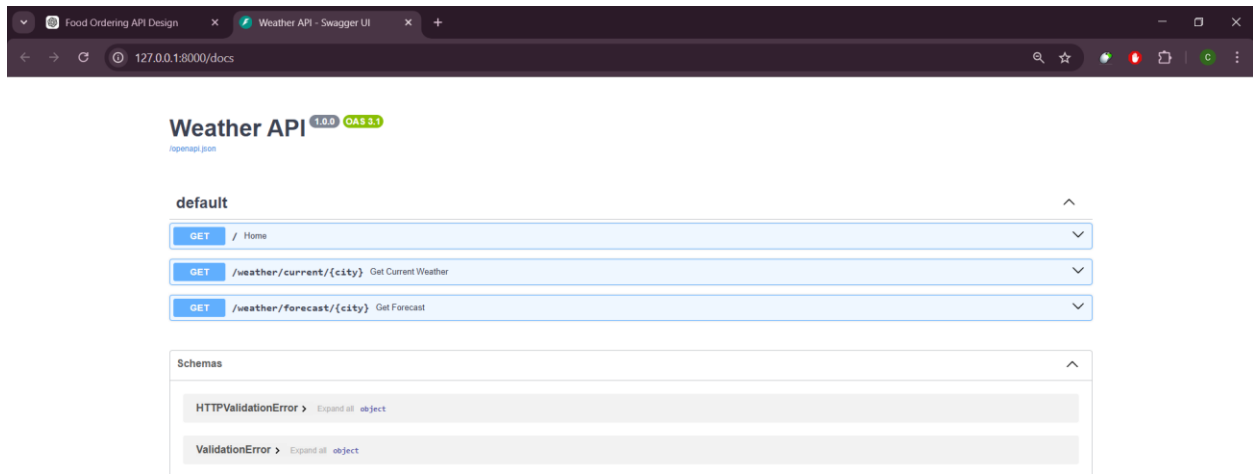
- Full working code
- Instructions to run
- Example API requests
- Suggested improvements (caching, rate limiting)

Code:

```
1 from fastapi import FastAPI, HTTPException
2 import requests
3 import os
4
5 app = FastAPI(title="Weather API", version="1.0.0")
6
7 # Add your API key here
8 API_KEY = "YOUR_API_KEY"
9
10 BASE_URL = "http://api.openweathermap.org/data/2.5"
11
12 # ===== Root =====
13
14 @app.get("/")
15 def home():
16     return {"message": "Weather API Running"}
17
18 # ===== Current Weather =====
19
20 @app.get("/weather/current/{city}")
21 def get_current_weather(city: str):
22     """
23     Fetch current weather for a city
24     """
25     url = f"{BASE_URL}/weather?q={city}&appid={API_KEY}&units=metric"
26     response = requests.get(url)
27
28     if response.status_code != 200:
29         raise HTTPException(status_code=404, detail="City not found")
30
31     data = response.json()
32
33     result = {
34         "city": city,
35         "temperature": data["main"]["temp"],
36         "humidity": data["main"]["humidity"],
37         "condition": data["weather"][0]["description"]
38     }
39
40     return result
```

```
21 def get_current_weather(city: str):
22     """
23     Fetch current weather for a city
24     """
25     url = f"{BASE_URL}/weather?q={city}&appid={API_KEY}&units=metric"
26     response = requests.get(url)
27
28     if response.status_code != 200:
29         raise HTTPException(status_code=404, detail="City not found")
30
31     data = response.json()
32
33     result = {
34         "city": city,
35         "temperature": data["main"]["temp"],
36         "humidity": data["main"]["humidity"],
37         "condition": data["weather"][0]["description"]
38     }
39
40     return result
41
42 # ===== Forecast =====
43
44 @app.get("/weather/forecast/{city}")
45 def get_forecast(city: str):
46     """
47     Fetch 5-day weather forecast
48     """
49     url = f"{BASE_URL}/forecast?q={city}&appid={API_KEY}&units=metric"
50     response = requests.get(url)
51
52     if response.status_code != 200:
53         raise HTTPException(status_code=404, detail="City not found")
54
55     data = response.json()
56
57     forecast_list = []
58
59     for item in data["list"][:5]: # first 5 entries
60         forecast_list.append({
61             "datetime": item["dt_txt"],
62             "temperature": item["main"]["temp"],
63             "humidity": item["main"]["humidity"],
64             "condition": item["weather"][0]["description"]
65         })
66
67     return {
68         "city": city,
69         "forecast": forecast_list
70     }
```

Output:



Explanation:

a Student Information API is developed to manage student records such as name, roll number, course, and year. It ensures unique roll numbers and provides endpoints to add, view, update, and delete student details, representing a university database system.

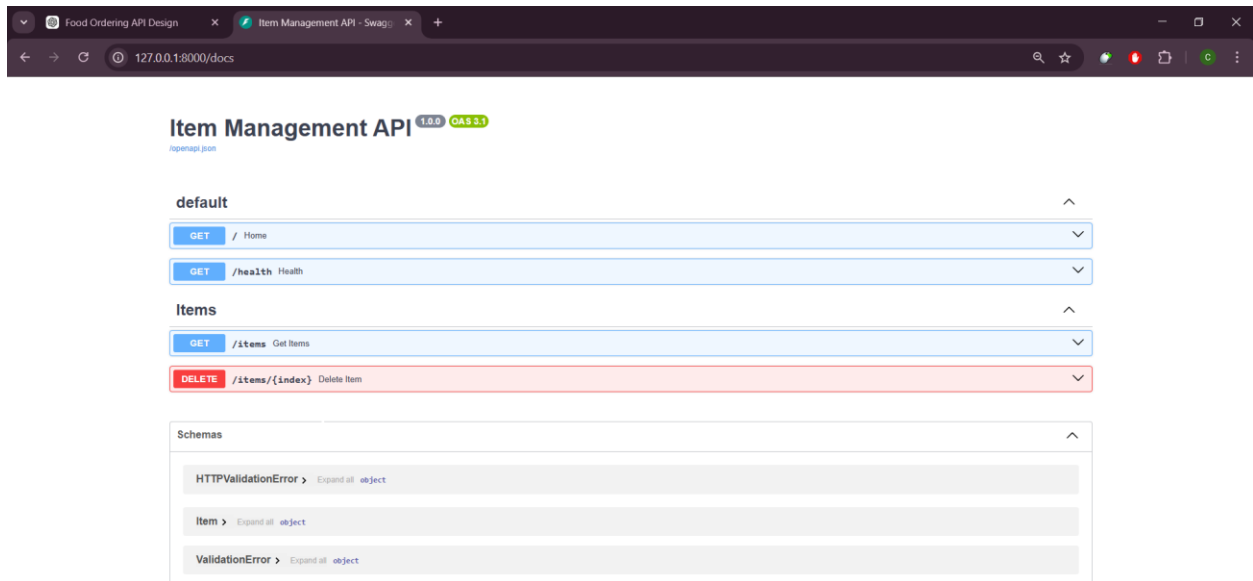
Task5:

Prompt: Act as a backend developer and create a DELETE endpoint for an API using Python (Flask preferred).
Requirements: * Endpoint: DELETE `/items/<int:index>` * Remove item from a list using index * Return deleted item in response * Handle errors (invalid index → 404) * Use proper JSON response
Output: * Full working Flask code * Sample input/output * Brief explanation Keep code simple and beginner-friendly.

Code:

```
1 from flask import Flask, jsonify
2
3 app = Flask(__name__)
4
5 # Sample data
6 items = [
7     {"name": "Notebook", "price": 250},
8     {"name": "Pen", "price": 20}
9 ]
10
11 # ===== GET Items =====
12 @app.route('/items', methods=['GET'])
13 def get_items():
14     return jsonify(items)
15
16 # ===== DELETE Item =====
17 @app.route('/items/<int:index>', methods=['DELETE'])
18 def delete_item(index):
19     """
20     Delete item by index
21     """
22     if index < 0 or index >= len(items):
23         return jsonify({"error": "Item not found"}), 404
24
25     removed_item = items.pop(index)
26
27     return jsonify({
28         "message": "Item deleted",
29         "item": removed_item
30     })
31
32 # ===== Run =====
33 if __name__ == "__main__":
34     app.run(debug=True)
```

Output:



Explanation:

extends functionality by integrating an external API, where a Weather Information API fetches real-time and forecast weather data from OpenWeatherMap. It processes and returns structured information like

temperature, humidity, and weather conditions, showcasing how third-party services can be integrated into backend systems. Finally